

UT-08: COLECCIONES DE DATOS: LISTAS, PILAS Y COLAS.

¿Qué vamos a ver?

- Colecciones de datos.
- Tipos de colecciones: listas, pilas, colas, tablas.
- Operaciones con colecciones.
 - Jerarquías de colecciones.
 - Acceso a elementos.
 - Recorridos. Iteradores.
 - Ordenación de listas
- Clases para la lectura de documentos de intercambio de datos.
- Uso de clases y métodos genéricos.
- Otras colecciones: interfaces map y set.

1. COLECCIONES DE DATOS

A menudo necesitamos guardar información, pero no sabemos de antemano el espacio que va a ocupar en memoria. En estos casos, los arrays no son la solución adecuada, ya que su tamaño debe permanecer fijo una vez creadas. Al redimensionarlas, lo que hacemos es crear otras nuevas y copiar todos los datos de la antigua, con la sobrecarga que ello supone para la gestión de memoria. En su lugar, necesitamos estructuras dinámicas de datos, es decir, objetos que alberguen datos que se pueden insertar y eliminar, cambiando el tamaño de la estructura sin alterar los datos restantes, todo ello en tiempo de ejecución.

Una colección es un objeto que sirve para agrupar un conjunto de objetos, llamados elementos, que generalmente guardan una relación entre ellos. Los métodos de las colecciones nos permiten llevar a cabo distintas operaciones con sus elementos, como inserción, eliminación búsqueda y ordenación.

2. TIPOS DE COLECCIONES: LISTAS, PILAS, COLAS Y TABLAS

Hasta el momento hemos visto un único tipo de colecciones, las tablas, que se trata de un contenedor de datos estático.

A partir de ahora vamos a conocer distintos tipos de colecciones dinámicos, que nos permitirán gestionar de una forma mucho más eficiente la memoria y el almacenamiento de los datos. Estos tipos de almacenamiento dinámica se denominan TAD o Tipos Abstractos de Datos.

En los siguientes apartados nos vamos a centrar en cómo se construyen tres tipos de TADs:

- Listas
- Pilas
- Colas

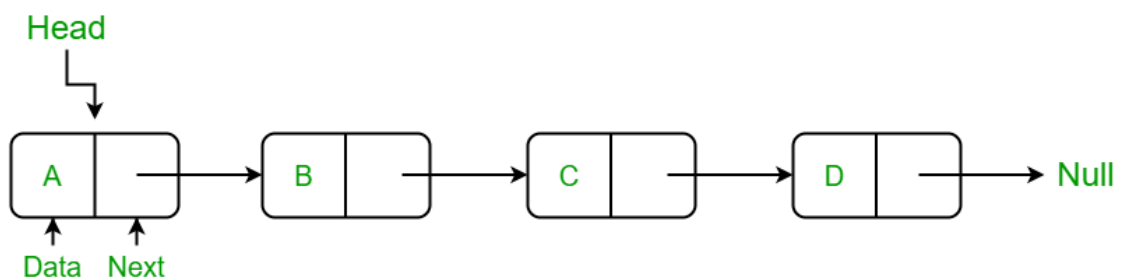
2.1. Listas

Responden a la necesidad de manejar sucesiones de datos que pueden estar repetidos y cuyo orden puede ser relevante. De alguna forma sustituyen a las tablas, con la diferencia de que podemos insertar y eliminar datos en ellas sin limitaciones de espacio. A cada elemento que compone una lista lo denominamos nodo, e incluye los datos a almacenar y los punteros necesarios.

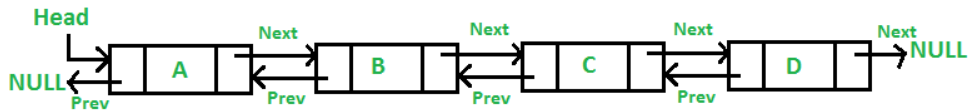


Dentro de las listas nos podemos encontrar con distintos tipos en función de cómo están enlazados sus elementos y cómo se pueden recorrer:

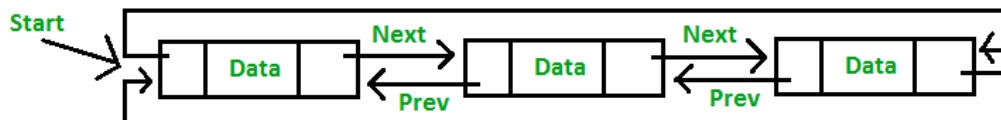
- **Listas enlazadas simples:** cada nodo apunta al siguiente. El último nodo apunta a nulo.



- **Listas doblemente enlazadas:** cada nodo apunta al siguiente y al anterior. En el caso del primer nodo el anterior apunta a nulo y en el caso del último nodo el siguiente apunta a nulo.



- **Listas circulares:** cada nodo apunta al siguiente y al anterior. El primer nodo apunta al último y al segundo. El último nodo apunta al primero y al penúltimo.



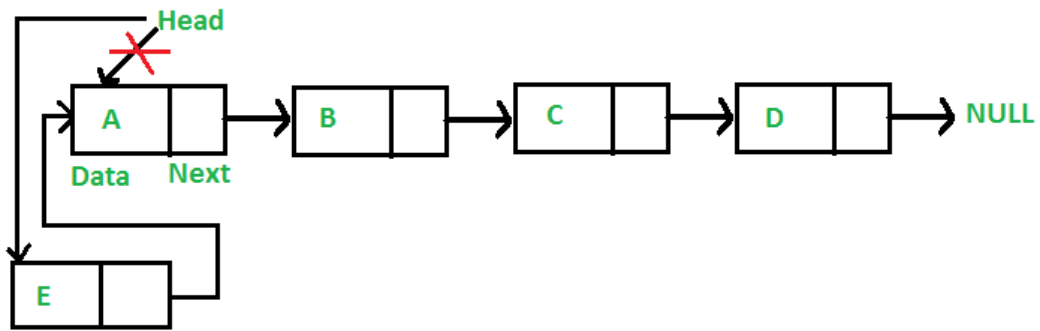
Las listas siempre necesitarán una variable que almacene de forma permanente la dirección del primer nodo, a esto lo llamaremos “cabeza”.

En esta unidad nos vamos a centrar en las listas enlazadas simples.

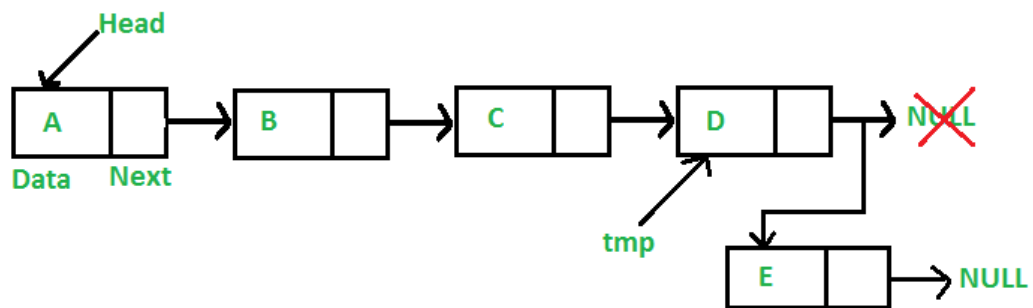
Con las listas (cualquiera de los tres tipos) podremos realizar las siguientes operaciones:

- Creación: establecemos la cabeza de la lista a nulo.
- Recorrido: utilizaremos una variable temporal para apuntar a la posición actual en la que nos encontramos de la lista.
- Inserción

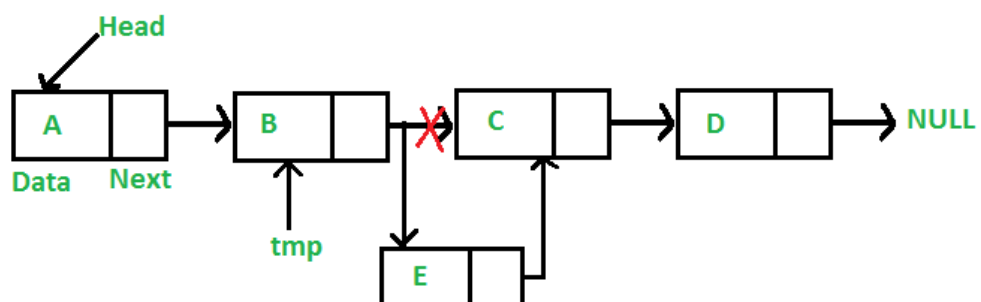
- Inserción en la primera posición (solo listas no ordenadas)



- Inserción en la última posición (solo listas no ordenadas)



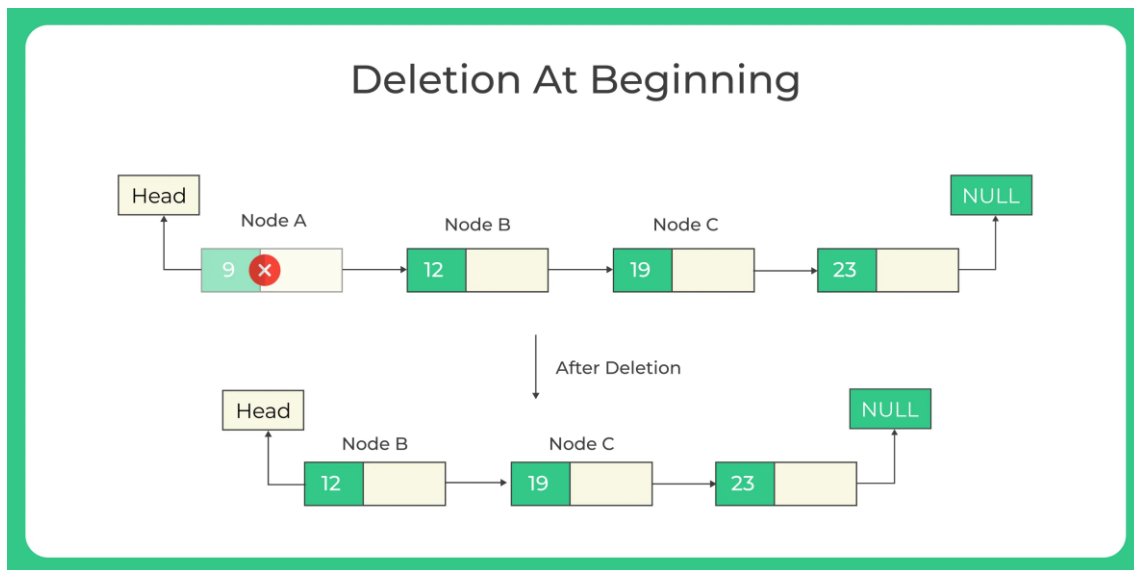
- Inserción en una posición determinada (solo listas no ordenadas)



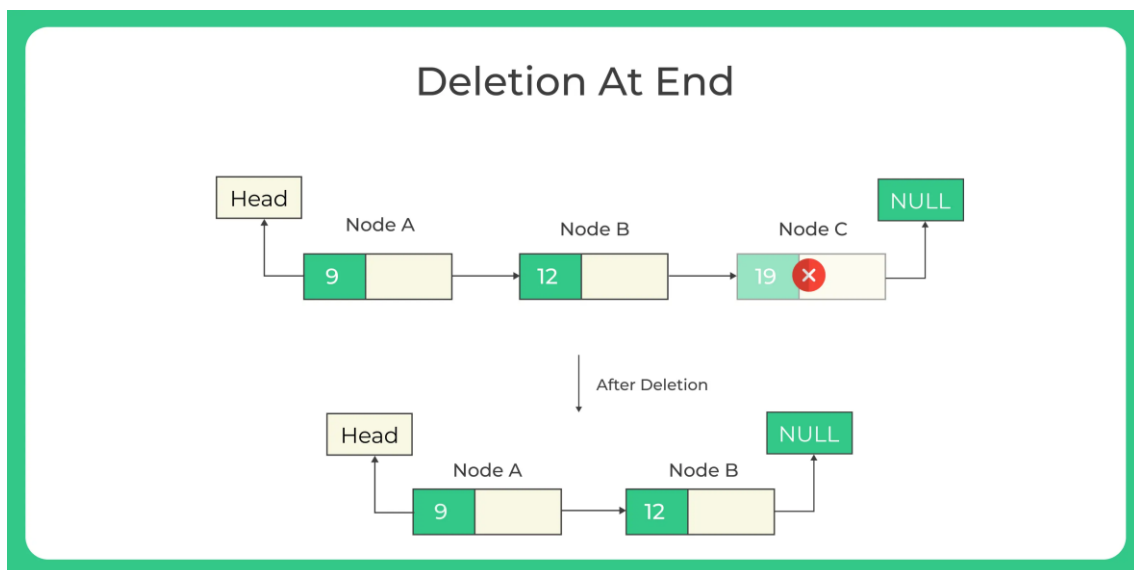
- Inserción ordenada (solo lista ordenadas)

- Eliminación

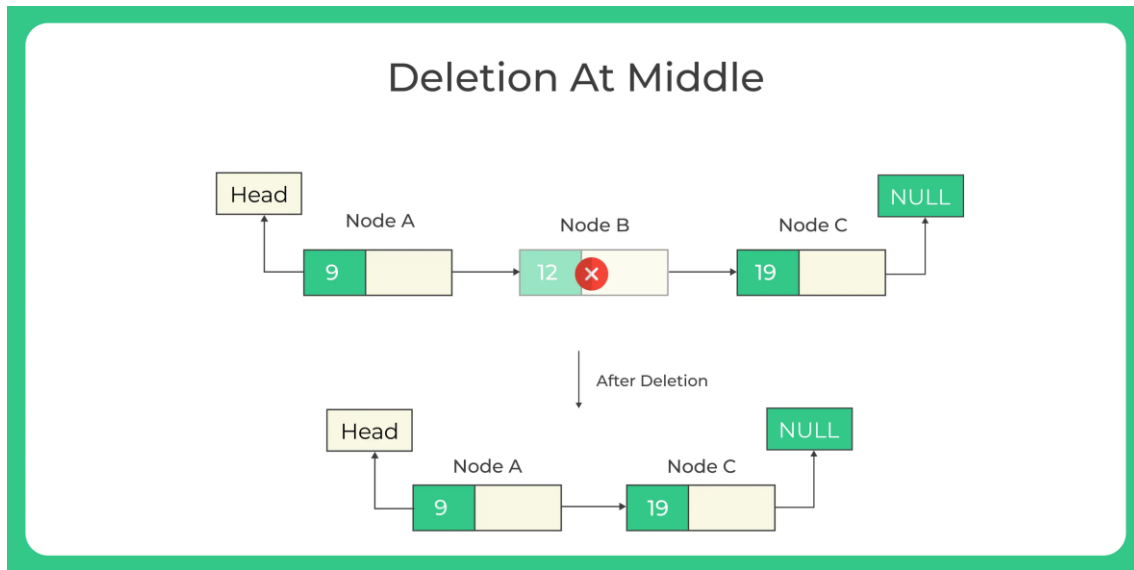
- Eliminación de la primera posición



- Eliminación de la última posición



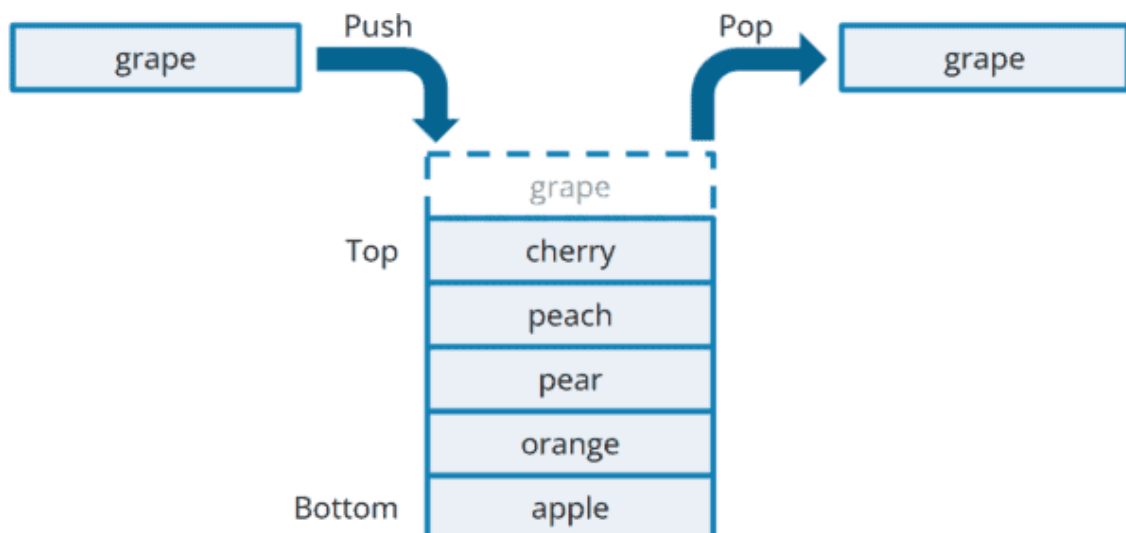
- Eliminación en una posición determinada



- Eliminación de un elemento determinado

2.2. Pilas

Las pilas son un tipo de colección similar en funcionamiento a la lista, pero con la particularidad de que sigue el máxima LIFO (Last In First Out), es decir, el último elemento en entrar en la pila será el primero en salir de ella.





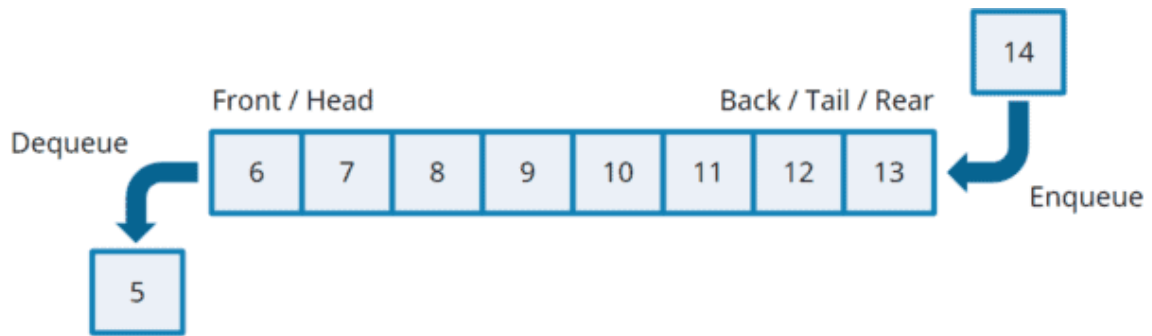
Esto se consigue de la siguiente manera:

- Realizando las inserciones únicamente en la cabecera. A esto se le denomina Apilar o Push.
- No nos será posible acceder a otro dato que no sea el de cabecera.
- Solo se podrá eliminar el nodo de la cabecera. A esto se le denomina Desapilar o Pop.

Por tanto, a la hora de implementar nuestra propia pila de almacenamiento solo tendremos que crear una estructura como la de la lista, pero solo nos centraremos en la inserción y la eliminación en la primera posición.

2.3. Colas

Al igual que ocurre con las pilas, las colas tienen un funcionamiento similar a las listas, pero con una determinada restricción... siguen el comportamiento FIFO (First In First Out), es decir, el primer elemento en entrar en la cola será el primero en salir de la cola.



Esto se consigue de la siguiente manera:

- Además de una variable que almacene la posición de la cabecera, necesitaremos una para la cola.
- Realizando las inserciones únicamente en la cola. A esto se le denomina Encolar o Queue.
- No nos será posible acceder a otro dato que no sea el de cabecera.
- Solo se podrá eliminar el nodo de la cabecera. A esto se le denomina Desencolar o Dequeue.

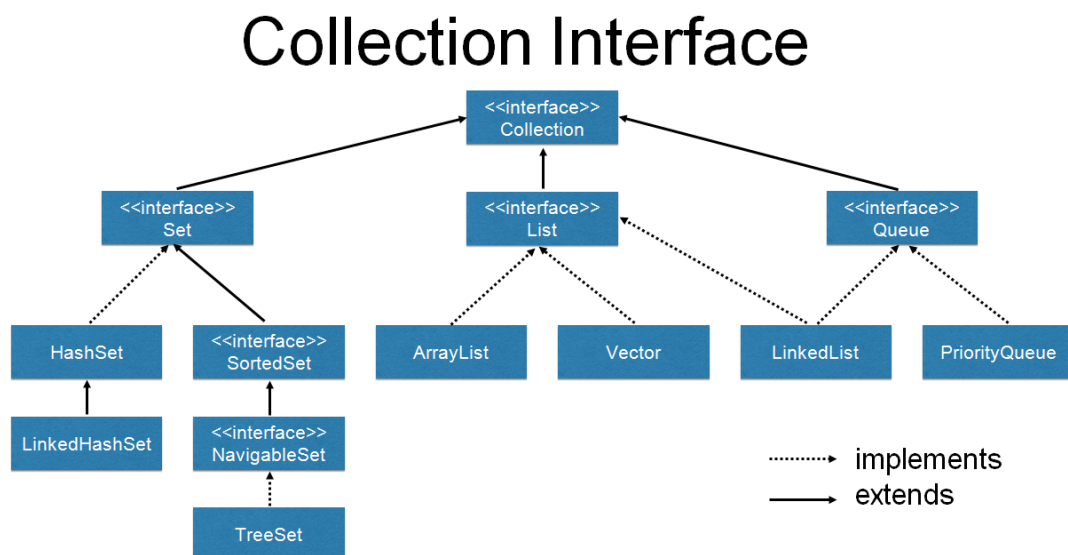
3. OPERACIONES CON COLECCIONES

Java proporciona una serie de estructuras dinámicas que comparten un conjunto de métodos declarados en la interfaz Collection.

3.1. Jerarquías de colecciones

Hay colecciones de diferentes tipos, adaptadas a distintos fines más o menos específicos. Por eso no existe una clase colección, sino todo un marco de trabajo (framework), con una estructura jerárquica de interfaces que se implementan en distintas clases, y con un conjunto de algoritmos que permiten la manipulación de los datos almacenados en las colecciones.

Todas ellas son dinámicas, ya que permiten añadir y quitar unidades de información (objetos llamados nodos o elementos) en tiempo de ejecución, sin más límite que la memoria del ordenador. Nosotros llamaremos colecciones a todas las clases que implementen la interfaz Collection, aunque también usaremos el nombre del tipo especial de colección: lista, conjunto o cola.



En los apartados consecutivos, vamos a centrarnos en las listas (List), más concretamente en los ArrayList.

3.2. Acceso a elementos

En primer lugar, debemos tener en cuenta que las listas con un tipo de estructura de datos genéricos, es decir, podemos introducir cualquier tipo de dato dentro de ellas, de hecho, la definición de las listas y arraylist es la siguiente:

List<T>

ArrayList<T>

Después, debemos saber que List<T> es una interfaz y, por tanto, no podemos implementar objetos de tipos List<T>, pero sí objetos de clases que implementan la interfaz List<T> como es el caso de ArrayList<T>, y por tanto a la hora de crear una lista podemos hacerlo de dos formas:

```
List<tipoDato> lista = new ArrayList<>();  
ArrayList<tipoDato> lista = new ArrayList<>();
```

Una vez que hayamos creado nuestra lista vacía, debemos ir introduciendo los datos. Para ello vamos a utilizar el método add, ya sea para insertarlo al final de la lista con add(dato), o en una posición determinada de la lista con add(posición, dato):

```
List<String> lista = new ArrayList<>();  
  
lista.add("10");  
lista.add("25");  
lista.add("30");  
lista.add(1, "40");  
  
//ahora lista estará compuesto por {"10", "40", "25", "30"}
```

Si queremos acceder a cada uno de los elementos podemos hacerlo por su posición dentro de la lista mediante el método get(posición):

```
lista.get(1); //devolverá "40"  
lista.get(0); // devolverá "10"  
lista.get(2); // devolverá "25"
```

Si necesitamos modificar el dato de que ocupa una determinada posición vamos a utilizar el método `set(posición, nuevoDato)`:

```
lista.set(1, "55");  
//ahora lista estará compuesto por {"10", "55", "25", "30"}
```

Para localizar elementos dentro de una lista tenemos varias opciones: `indexOf(dato)` nos devolverá el índice de la primera aparición del dato, `lastIndexOf(dato)` nos devolverá el índice de la última aparición del dato y `contains(dato)` nos indicará si ese dato se encuentra en la lista.

```
lista.indexOf("25"); // {"10", "55", "25", "30"} devolverá 2  
lista.lastIndexOf("30");// {"30", "55", "25", "30"} devolverá 3  
lista.contains("75"); // {"10", "55", "25", "30"} devolverá false
```

Si queremos eliminar uno de los elementos de la lista podemos hacerlo mediante distintas variantes del método `remove`, por la posición... `remove(posición)`, ... o la primera aparición de un elemento `remove(dato)`:

```
lista.remove(0);  
lista.remove("30");  
//ahora lista estará compuesto por {"55", "25"}
```

Si queremos eliminar todos los elementos de una lista, para dejarla vacía, utilizaremos el método `clear()`.

```
lista.clear();  
//ahora la lista está vacía
```

Para comprobar si una lista está vacía utilizamos el método `isEmpty()`.

```
lista.isEmpty();  
//devolverá true su está vacía, false en caso contrario
```

Si deseamos conocer el número de elementos que componen la lista utilizaremos el método `size()`.

```
lista.size();  
// {"10", "55", "25", "30"} devolverá 4
```

En la ayuda oficial de Java disponéis de algunos métodos más bastante útiles:

<https://docs.oracle.com/javase/8/docs/api/java/util/List.html>

3.3. Recorridos. Iteradores

Podríamos recorrer las listas con cualquiera las distintas instrucciones de bucle que ya vimos en su momento, pero lo más recomendable es utilizar for, ya sea en su variante normal con una variable como contador o con la variante que denominamos foreach. Veamos dos ejemplos que serían equivalentes.

```
TipoDato objeto;  
int max = lista.size();  
for(int i = 0; i < max; i++){  
    objeto = lista.get(i);  
    ...  
}
```

La variante del for con contador nos puede servir para recorrer una lista de forma parcial, en lugar de completo, mediante el control del contador.

En el for “clásico” vamos almacenar el tamaño en una variable, puesto que lista.size() es un método y sería ineficiente consulta en cada vuelta el tamaño si siempre es el mismo. Pero en el caso de que dentro del for vayamos a insertar o eliminar elementos en la lista, esto no nos sería útil.

```
for(TipoDato objeto : lista){  
    objeto ...  
}
```

El uso del foreach queda más elegante a la hora de recorrer una lista de forma completa.

Cuando necesitemos aumentar, o reducir, el contenido de la lista será más conveniente usar while y do while.

```
List<Integer> lista = new ArrayList<>();  
while(lista.size() < 10){  
    lista.add(0);  
}
```

3.4. Ordenación de listas

El interfaz List cuenta con un método denominado sort, al cual es necesario pasar como parámetro una clase que instancie el interfaz Comparator, que ya conocemos.

```
sort(Comparator <? Super> c)
```

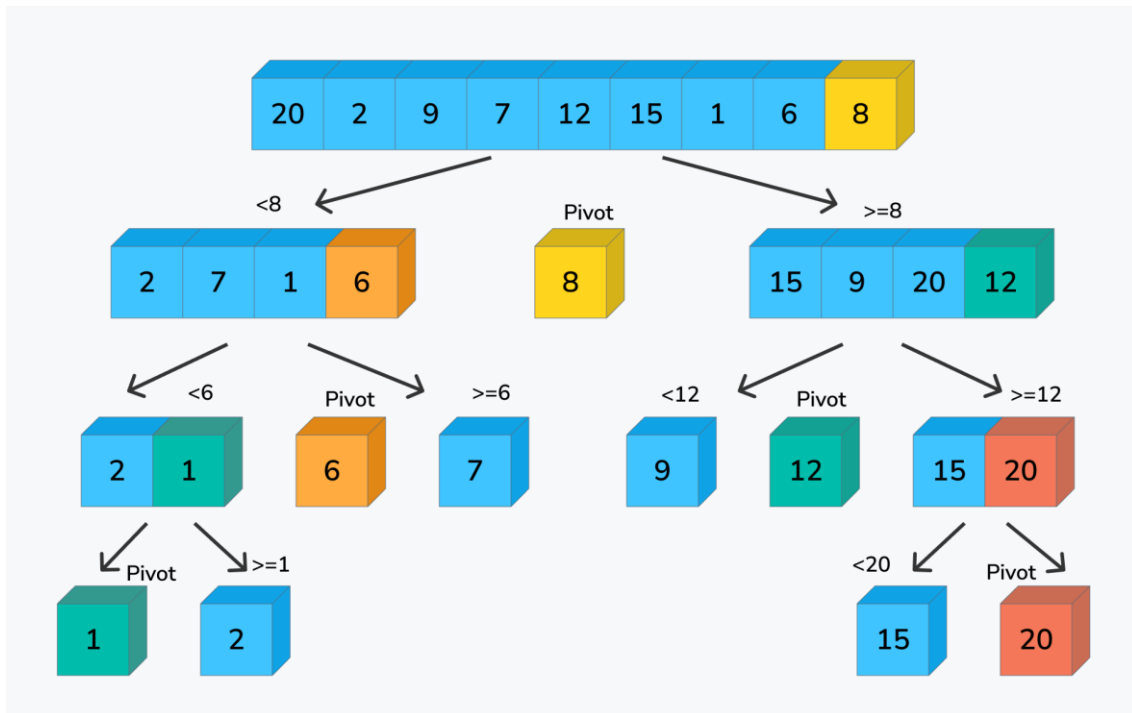
```
lista.sort(new ComparatorLista());  
  
o  
  
import java.util.Collections;  
  
Collections.sort(lista, new ComparatorLista());
```

De esta forma ordenará los distintos elementos de la lista en función de las comparaciones mediante el Comparator.

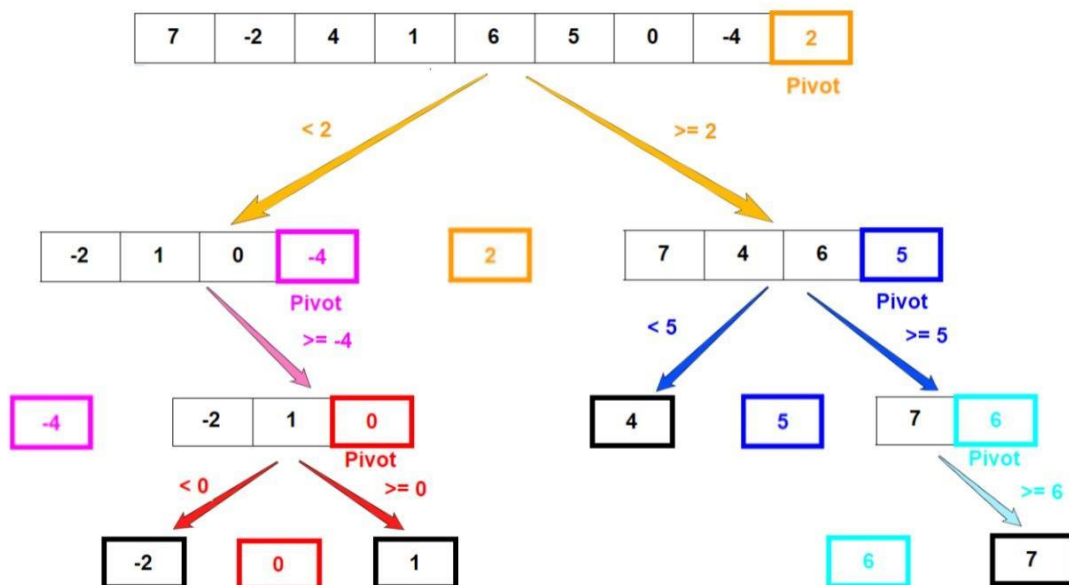
También existen otros modos de ordenación “manuales”, algo más avanzados a los que vimos en su momento con los arrays. Estos métodos de ordenación se basan en el concepto de “Divide y vencerás” y en la recursividad. Se llaman Quick Sort y Merge Sort.

3.4.1. Quick sort

Ejemplo 1:



Ejemplo 2:

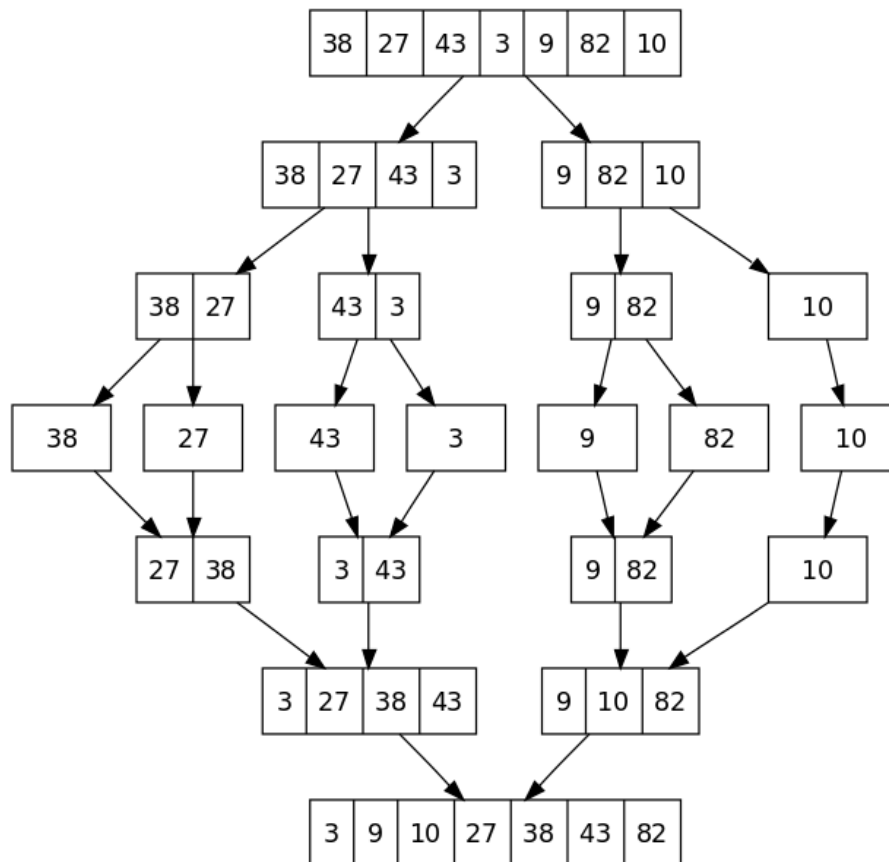


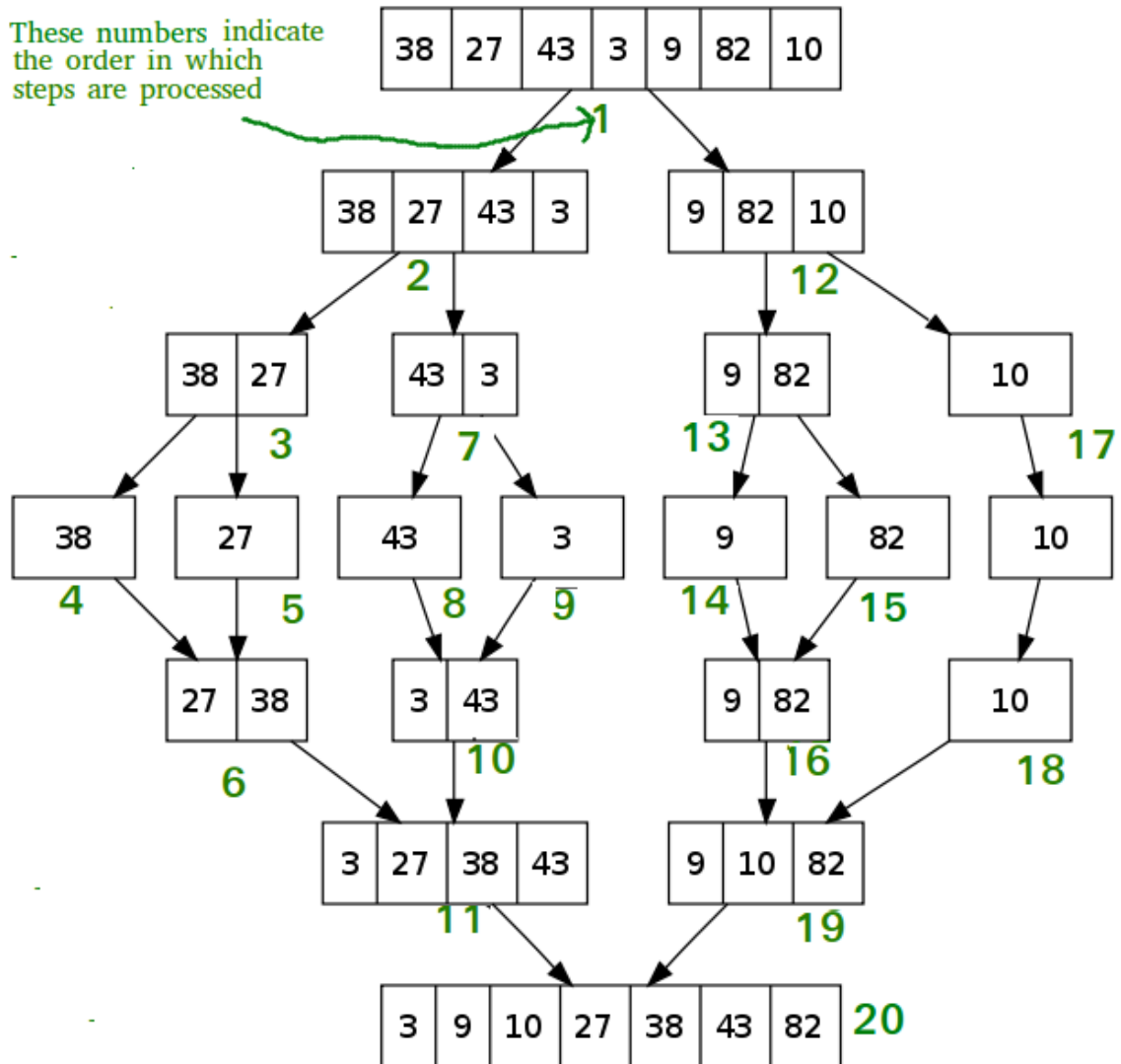
Quick sort se basa en determinar una posición pivote e ir dividiendo la lista en dos sublistas, una con los elementos menores al pivote y otra con los elementos mayores al pivote, quedándonos con este entre ambas sublistas. Después,

realizaremos la misma operación cada una de las sublistas, hasta quedarnos con listas de un solo elemento ordenadas de menos a mayor, y luego las unimos.

En el aula virtual disponéis de un vídeo explicativo de cómo funciona. Vamos a verlo...

3.4.2. Merge Sort





Merge sort se basa en ir dividiendo por la mitad la lista en secciones cada vez más pequeñas hasta que nos quedemos con varias listas de un solo elemento, y después la vamos a ir uniendo, por etapas, los distintos elementos en orden, hasta obtener la lista ordenada.

En el aula virtual disponéis de un vídeo explicativo de cómo funciona. Vamos a verlo...

4. CLASES PARA LA LECTURA DE DOCUMENTOS DE INTERCAMBIO DE DATOS

Existen una gran variedad de tipos de ficheros que se pueden utilizar para intercambiar información entre organizaciones por medios electrónicos, desde simples txt hasta otros muchos más complejos y encriptados. Algunos de los más conocidos actualmente son Json y XML. En nuestro caso nos vamos a centrar en estos últimos.

XML, siglas de lenguaje de marcado extensible, permite definir y almacenar datos de forma compartible. XML admite el intercambio de información entre sistemas de computación, como sitios web, bases de datos y aplicaciones de terceros. Las reglas predefinidas facilitan la transmisión de datos como archivos XML a través de cualquier red, ya que el destinatario puede usar esas reglas para leer los datos de forma precisa y eficiente.

Un archivo de lenguaje de marcado extensible (XML) es un documento basado en texto que se puede guardar con la extensión .xml.

Cualquier archivo XML incluye los siguientes componentes:

4.1. Estructura de los ficheros XML

4.1.1. Declaración XML

Un documento XML comienza con alguna información sobre el propio XML. Por ejemplo, podría mencionar la versión XML que sigue. Esta apertura se denomina declaración XML. A continuación se muestra un ejemplo:

```
<?xml version="1.0" encoding="UTF-8"?>
```

4.1.2. Elementos XML

Todas las demás etiquetas que se creen en un documento XML se denominan elementos XML. Los elementos XML pueden contener las siguientes características:

- Texto

- Atributos
- Otros elementos

Todos los documentos XML comienzan con una etiqueta principal, que se denomina elemento raíz.

Por ejemplo:

```
<ListaInvitación>
<familia>
  <tía>
    <nombre>Cristina</nombre>
    <nombre>Estefanía</nombre>
  </tía>
</familia>
</ListaInvitación>
```

<ListaInvitación> es el elemento raíz; familia y tía son otros nombres de elementos.

4.1.3. Atributos XML

Los elementos XML pueden tener otros descriptores denominados atributos. Puede definir sus propios nombres de atributos y escribir los valores de los atributos entre comillas, como se muestra a continuación.

```
<edad de la persona="22">
```

4.1.4. Contenido XML

Los datos de los archivos XML también se denominan contenido XML. Por ejemplo, en el archivo XML, es posible que veas datos como este.

```
<amigo>
  <nombre>Carlos</nombre>
  <nombre>Esteban</nombre>
</amigo>
```

Los valores de los datos Carlos y Esteban son el contenido.

4.2. Escritura de ficheros XML con Java

Para poder crear un fichero XML desde nuestro programa vamos a necesitar importar una gran cantidad de librería. A continuación tenemos el listado:

```
java.io.File;
javax.xml.parsers.DocumentBuilder;
javax.xml.parsers.DocumentBuilderFactory;
javax.xml.parsers.ParserConfigurationException;
javax.xml.transform.Result;
javax.xml.transform.Source;
javax.xml.transform.Transformer;
javax.xml.transform.TransformerException;
javax.xml.transform.TransformerFactory;
javax.xml.transform.dom.DOMSource;
javax.xml.transform.stream.StreamResult;
org.w3c.dom.Attr;
org.w3c.dom.DOMImplementation;
org.w3c.dom.Document;
org.w3c.dom.Element;
org.w3c.dom.Text;
```

Gracias a todas estas librerías seremos capaces de crear nuestros propios ficheros XML desde un programa escrito en Java. Veamos el procedimiento:

En primer lugar necesitaremos crear un objeto de tipo DocumentBuilderFactory mediante el método DocumentBuilderFactory.newInstance() y con el objeto resultado vamos a generar un objeto DocumentBuilder usando el método newDocumentBuilder(). Es importante que esto lo introduzcamos dentro de una estructura try...catch, de la siguiente manera.

```
public static void crearFicheroXML(){
    DocumentBuilderFactory factory =
    DocumentBuilderFactory.newInstance();
    DocumentBuilder builder;
    try {
        builder = factory.newDocumentBuilder();
        ...
    } catch (ParserConfigurationException | TransformerException ex) {
        System.out.println(ex.getMessage());
    }
```

```
}  
}
```

Después de esto vamos a necesitar crear un objeto `DOMImplementation` con el método `getDOMImplementation`. Gracias a este objeto ya podremos crear el documento XML en si con el método `createDocument`, al que le pasaremos el nombre de nuestro elemento raíz. Los otros dos parámetros los podemos dejar a null. También le indicaremos la versión de XML que vamos a utilizar, en nuestro caso la 1.0.

```
public static void crearFicheroXML(){  
    DocumentBuilderFactory factory =  
    DocumentBuilderFactory.newInstance();  
    DocumentBuilder builder;  
    try {  
        builder = factory.newDocumentBuilder();  
        DOMImplementation implementation =  
        builder.getDOMImplementation();  
  
        Document documento = implementation.createDocument(null,  
"elementoRaiz", null);  
        documento.setXmlVersion("1.0");  
        ...  
  
    } catch (ParserConfigurationException | TransformerException ex) {  
        System.out.println(ex.getMessage());  
    }  
}
```

Una vez que tenemos el documento con el nodo raíz, solo tenemos que ir añadiendo los distintos elementos con sus subelementos, atributos y contenido.

Para esto es importante que siempre tengamos en mente la estructura que deseamos construir dentro del documento.

Vamos a realizar un ejemplo sencillo para obtener una estructura como la que se muestra a continuación.

```
<?xml version="1.0" encoding="UTF-8" standalone="no"?>  
<elementoRaiz>  
    <elementoPadre>
```

```
<elementoHijo1 AtributoHijo1="ValorAtributoHijo1" />
<elementoHijo2>textoElementoHijo2</elementoHijo2>
<elementoHijo3>textoElementoHijo3</elementoHijo3>
</elementoPadre>
</elementoRaiz>
```

Creamos el elemento padre...

```
Element elementoPadre = documento.createElement("elementoPadre");
```

Creamos el elemento hijo 1 y le asignamos el atributo y el valor...

```
Element elemento1 = documento.createElement("elementoHijo1");
elemento1.setAttribute("AtributoHijo1", "ValorAtributoHijo1");
```

Creamos los elementos hijos 2 y 3, sus contenidos y unimos cada uno al que le corresponde...

```
Element elemento2 = documento.createElement("elementoHijo2");
Text textoElemento2 = documento.createTextNode("textoElementoHijo2");
elemento2.appendChild(textoElemento2);
```

```
Element elemento3 = documento.createElement("elemento3");
Text textoElemento3 = documento.createTextNode("textoElementoHijo3");
elemento3.appendChild(textoElemento3);
```

Después, añadimos los nodos hijos al nodo padre...

```
elementoPadre.appendChild(elemento1);
elementoPadre.appendChild(elemento2);
elementoPadre.appendChild(elemento3);
```

Añadimos el nodo padre al nodo raíz...

```
documento.getDocumentElement().appendChild(elementoPadre);
```

Finalmente, damos un nombre al nuevo fichero e introducimos el documento XML que hemos generado en dicho fichero.

```
Result result = new StreamResult(new File("elementos.xml"));
Source source = new DOMSource(documento);

Transformer transformer =
TransformerFactory.newInstance().newTransformer();
transformer.transform(source, result);
```

4.2. Lectura de ficheros XML con Java

Para leer un fichero XML, y cargarlo en memoria, a través de un programa codificado en Java también necesitaremos hacer uso de varias librerías:

```
java.io.File;
java.io.IOException;
javax.xml.parsers.DocumentBuilder;
javax.xml.parsers.DocumentBuilderFactory;
javax.xml.parsers.ParserConfigurationException;
org.w3c.dom.Document;
org.w3c.dom.Element;
org.w3c.dom.NamedNodeMap;
org.w3c.dom.Node;
org.w3c.dom.NodeList;
org.xml.sax.SAXException;
```

Para leer los documentos XML nos vamos a ayudar de los nombres de las etiquetas de los nodos, y a partir de ahí leeremos sus nodos hijos.

Veamos cómo leeríamos el fichero que hemos creado antes:

```
<?xml version="1.0" encoding="UTF-8" standalone="no"?>
<elementoRaiz>
  <elementoPadre>
    <elementoHijo1 AtributoHijo1="ValorAtributoHijo1" />
    <elementoHijo2>textoElementoHijo2</elementoHijo2>
    <elementoHijo3>textoElementoHijo3</elementoHijo3>
  </elementoPadre>
</elementoRaiz>
```

Lo primero que vamos a hacer, al igual que con la escritura es crear los objetos DocumentBuilderFactory y DocumentBuilder.

```
public static void crearFicheroXML(){
```

```
DocumentBuilderFactory factory =
DocumentBuilderFactory.newInstance();
DocumentBuilder builder;
try {
    builder = factory.newDocumentBuilder();
    ...

} catch (ParserConfigurationException | SAXException | IOException
ex){
    System.out.println(ex.getMessage());
}
}
```

Después abrimos el fichero xml que vamos a leer...

```
Document documento = builder.parse(new File("elementos.xml"));
```

Buscamos todos los nodos cuya etiqueta sea elementoPadre...

```
NodeList listaPadres = documento.getElementsByTagName("elementoPadre");
```

En nuestro caso solo hay uno, pero al almacenarlo en una lista podemos almacenar un número indeterminado de ellos y luego ir examinando uno a uno.

A continuación, por cada uno de los nodos que nos ha devuelto, vamos a comprobar que es un nodo del tipo Element, y si lo es extraeremos sus hijos.

```
for(int i = 0; i < listaPadres.getLength(); i++){
    Node nodo = listaPadres.item(i);
    if(nodo.getNodeType() == Node.ELEMENT_NODE){
        Element e = (Element) nodo;
        System.out.println("Elemento: " + nodo.getNodeName());
        NodeList hijos = e.getChildNodes();
        ...
    }
}
```

Como sabemos que unos elementos tienen atributos y otros contenido, vamos a preguntar a cada nodo hijo por el número de atributos que tiene. Si tiene más de 0 sabemos que es un elementoHijo1, si no, es de lo otros.

```
for (int j = 0; j < hijos.getLength(); j++){
    Node hijo = hijos.item(j);
```



```
if(hijo.getNodeType() == Node.ELEMENT_NODE){
    NamedNodeMap atributos = hijo.getAttributes();
    if(atributos.getLength() > 0){
        Node atributo = atributos.getNamedItem("AtributoHijo1");
        System.out.println("NodoHijo: " + hijo.getNodeName() + ",
Atributo: " + atributo.getNodeValue());
    }
    else{
        System.out.println("NodoHijo: " + hijo.getNodeName() + ",
Valor: " + hijo.getTextContent());
    }
}
}
```

El resultado que nos devolvería por consola al ejecutar este procedimiento de lectura sería el siguiente:

Elemento: elementoPadre

NodoHijo: elementoHijo1, Atributo: ValorAtributoHijo1

NodoHijo: elementoHijo2, Valor: textoElementoHijo2

NodoHijo: elementoHijo3, Valor: textoElementoHijo3

5. USO DE CLASES Y MÉTODOS GENÉRICOS

En la unidad ya vimos cómo podemos crear y utilizar nuestras propias clases e interfaces genéricas, con sus métodos correspondientes. Si embargo, dentro de cualquier clase, tanto si está definida con tipos genéricos como si no, podemos implementar métodos con sus propios parámetros genéricos, distintos de los que pueda tener la clase. Dichos métodos se llaman métodos genéricos.

Veamos un ejemplo...

El siguiente método está preparado para que nos devuelva el número de elementos nulos que hay en un array que se le pasa como parámetro. El tipo T de la tabla es genérico, y se declara en la definición del método, justo antes del tipo devuelto.

```
static <T> int numeroDeNulos(T[] tabla){
    int cont = 0;
    for( T elem : tabla){
        if(elem == null){
            cont++;
        }
    }
    return cont;
}
```

Este método se podría incluir en cualquier clase y depende en nada de los parámetros propios de la misma.

El tipo asociado a un método genérico también puede estar limitado. Por ejemplo, si quisiéramos que nuestro método `numeroDeNulos()` solo funcionara para arrays de objetos tipos persona o descendientes de la misma pondríamos `<T extends Persona>` en vez de `<T>`.

```
static <T extends Persona> int numeroDeNulos(T[] tabla){
    int cont = 0;
    for( T elem : tabla){
        if(elem == null){
            cont++;
        }
    }
    return cont;
}
```

6. OTRAS COLECCIONES: INTERFACES MAP Y SET.

6.1. Interfaz Set

La interfaz Set trata los datos como un conjunto matemático, eliminando las repeticiones y sin un orden establecido; aunque, como veremos, una de sus implementaciones permite introducir un orden. Todos sus métodos son herencia de Collection. Lo único que añade es la restricción de no permitir duplicados. Esto significa que si intentamos insertar un elemento que ya existe, no lo hará.

Los métodos más utilizados ya los conocemos, puesto que también son utilizados por la interfaz List. Esto es debido a que son métodos heredados de Collection:

- `int size()`
- `boolean isEmpty()`
- `boolean contains(Object element)`
- `boolean add(E element)`
- `boolean remove(E element)`
- `boolean addAll(Collection <? Extends E>c)`
- `void clear()`

Para conocer todos los métodos de los que dispone esta interfaz podéis consultarlos en el siguiente link de la ayuda oficial:

<https://docs.oracle.com/javase/8/docs/api/java/util/Set.html>

Para recorrer un conjunto se recomienda el uso de `foreach`.

Las diferencias más importantes son el orden en que se van insertando los elementos nuevos y que un elemento que ya está en el conjunto no se puede volver a insertar, ya que no son posibles los elementos repetidos. Cuando intentamos insertar un elemento repetido con un método `add()` o con `addAll()`, no se producirá ningún error ni se arrojará ninguna excepción; sencillamente, el elemento no se inserta y, como el conjunto no habrá cambiado, el método devuelve `false`.

Sin embargo, no tendremos los métodos propios de listas, que vienen declarados en la interfaz List. En particular, no es posible el acceso posicional por medio de índices, aunque sí las iteraciones.

Las implementaciones de Set son las clases: `HashSet`, `TreeSet` y `LinkedHashSet`.

- **HashSet**: tiene un buen rendimiento, aunque no garantiza ningún orden en la inserción.
- **TreeSet**: a pesar de tener peor rendimiento, garantiza el orden basado en el valor de los elementos insertados. El criterio de ordenación por defecto es el natural (el proporcionado por el método `compareTo()` de la clase genérica `E`) o bien se especifica por medio de un comparador pasado como parámetro al constructor.
- **LinkedHashSet**: inserta los elementos al final, con lo cual se garantiza un orden basado en la inserción.

Al contrario de lo que pasa con las listas, las tres implementaciones de `Set` tienen diferencias en su comportamiento. Es muy común utilizar variables de tipo `Set` para referenciarlos. Esto permite aprovechar el polimorfismo de las distintas implementaciones y hacer transformaciones de unas en otras

Veamos un ejemplo de `TreeSet`, en el que la clase `cliente` tiene un método `compareTo()` basado en el atributo `ID`:

```
TreeSet<Cliente> conjuntoClientes = new TreeSet<>();  
  
conjuntoClientes.add(new Cliente("111", "Marta", "2004"));  
conjuntoClientes.add(new Cliente("115", "Jorge", "2003"));  
conjuntoClientes.add(new Cliente("112", "Carlos", "2002"));  
System.out.println(conjuntoClientes);
```

Veremos por pantalla:

```
[ID: 111 Nombre: Marta Edad: 20,  
  ID: 112 Nombre: Carlos Edad: 18,  
  ID: 115 Nombre: Jorge Edad: 21  
]
```

Observamos que el orden en el que aparecen no coincide con el orden de inserción, sino que están ordenados por `ID` creciente.

Si ahora intentamos volver a insertar uno de los elementos anteriores, por ejemplo, el de Marta...

```
boolean insertado = conjuntoClientes.add(new Cliente("111", "Marta",  
"2004"));  
System.out.println(insertado);  
System.out.println(conjuntoClientes);
```

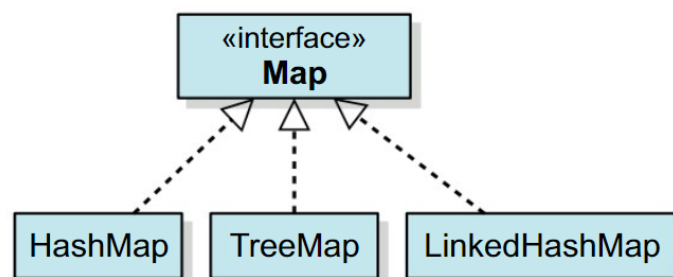
Aparece por pantalla...

```
false  
[ID: 111 Nombre: Marta Edad: 20,  
ID: 112 Nombre: Carlos Edad: 18,  
ID: 115 Nombre: Jorge Edad: 21  
]
```

donde vemos que la inserción ha devuelto un false (no ha insertado) y que el conjunto no ha cambiado. Pero si queremos otro conjunto de clientes ordenados por nombre, lo creamos pasando a su constructor, como argumento, un comparador basado en el atributo nombre.

6.2. Interfaz Map

Los mapas o diccionarios son estructuras cuyos elementos, a los que llamaremos entradas (objetos del tipo Map.Entry), son pares clave/valor en vez de valores individuales como en las colecciones. Todas ellas implementan la interfaz Map, que no hereda de Collection. Por tanto, los mapas no son colecciones, aunque están íntimamente relacionados con ellas y funcionan dentro del mismo entorno de trabajo. Vamos a conocer tres implementaciones de Map: HashMap, TreeMap y LinkedHashMap, se diferencian entre sí de forma similar a HashSet, TreeSet y LinkedSet.



En un mapa se insertan entradas que constan de una clave, que no se puede repetir, y un valor asociado con ella, que sí puede estar repetido.

Las operaciones fundamentales en un mapa son la inserción, la lectura y la eliminación de entradas.

Como ejemplo del uso de mapas vamos a utilizar la implementación HashMap, que no garantiza ningún orden de inserción de las entradas, aunque es muy eficiente en cuanto a la velocidad de acceso a los datos. El constructor que vamos a utilizar es el que se define de la siguiente manera:

```
Map<K, V> m = new HashMap<>();
```

Donde K es el tipo de la clave y V el de los valores. Son tipos genéricos que, necesariamente, serán clases o interfaces y no tipos primitivos. Por tanto, si queremos crear una instancia de un HashMap con una clave de tipo String y valores de tipo Double, lo haremos de la siguiente manera:

```
Map<String, Double> m = new HashMap<>();
```

Para insertar elementos en este mapa vamos a utilizar el método

```
V put(K clave, V valor)
```

En el caso de que no hubiese ningún valor asociado a esa clave anteriormente, se insertará el par y el método devolverá null. En el caso de que ya existiese la clave, se modificará el valor por el nuevo y se devolverá el antiguo.

```
m.put("Ana", 1.65);  
m.put("Marta", 1.60);  
m.put("Luis", 1.73);  
m.put("Pedro", 1.69);  
System.out.println(m);
```

Esto devolverá por pantalla:

```
{Marta=1.6, Ana=1.65, Luis=1.73, Pedro=1.69}
```

Si queremos cambiar la estatura de Pedro, insertamos otra vez una entrada con la misma clave y el nuevo valor.

```
m.put("Pedro", 1.71);  
System.out.println(m);
```

Y obtenemos...

```
{Marta=1.6, Ana=1.65, Luis=1.73, Pedro=1.71}
```

Para eliminar una entrada del mapa disponemos de

```
V remove (Object k)
```

que elimina la entrada cuya clave es k, si existe. En este caso, devuelve el valor asociado a la clave. En caso contrario, devuelve null.

```
m.remove("Pedro");  
System.out.println(m);
```

Devolvería

```
{Marta=1.6, Ana=1.65, Luis=1.73}
```

Para eliminar todas las entradas utilizamos clear().

```
m.clear();
```

Para obtener el valor de una entrada utilizamos, get que devuelve el valor asociado a la clave k o null si no hay ninguna entrada con esa clave.

```
V get (Object k);
```

```
m.get("Ana") // 1.65
```

Finalmente, si queremos saber si una clave se encuentra en el mapa utilizaremos containsKey que nos devolverá true o false en función de si dentro del mapa hay una entrada con la clave k.

```
boolean containsKey(Object k);
```

```
if(containsKey("Ana"){  
    System.out.println(m.get("Ana"));  
}  
// Escribirá por pantalla 1.65
```

Para conocer más operaciones que podemos realizar con los Maps, puedes consultar el siguiente link de la ayuda oficial:

<https://docs.oracle.com/javase/8/docs/api/java/util/Map.html>